Software Quality Engineering (SQE)

Mids Quick Notes: short definitions + examples | Lectures 1 to 4

Lecture 1: Software Quality and Quality Assurance

Term	Short definition	Example
Software quality	Degree to which software meets its stated requirements and the implicit expectations of professional software.	WhatsApp must send messages (explicit) and also be fast, secure and not crash (implicit).
Explicit requirements	What the specification directly states.	'User can log in with email and password.'
Documented standards	Coding, design and process standards the team commits to.	Every pull request must be code-reviewed.
Implicit expectations	Unwritten traits users assume.	Login page loads quickly; password is hidden.
Quality of design	How well requirements, architecture and specs were conceived before coding.	An architect's house plan.
Quality of conformance	How closely the built product matches the approved design.	The builder follows the plan without shortcuts.

Why quality matters (5 reasons)

Term	Short definition	Example
Customer trust	Reliable software keeps users and reduces churn.	Users leave a wallet app that keeps failing payments.
Lower cost of ownership	Fewer defects mean less rework, support and emergency fixes.	No 3 a.m. production bug-fix calls.
Brand and reputation	Public failures damage credibility.	2024 CrowdStrike faulty update disrupted airlines and banks worldwide.
Safety and compliance	Many domains legally need verified software.	Medical devices, aviation, banking.
Competitive advantage	Consistent quality differentiates a product.	Users keep the app that crashes less.

Six dimensions of quality (F-R-U-E-M-P), ATM example

Term	Short definition	Example
Functionality	Performs required tasks correctly and completely.	ATM dispenses the exact amount requested.
Reliability	Runs consistently without unexpected failures or data loss.	If a transaction fails, money is not deducted.
Usability	Easy for intended users to learn and operate.	An elderly person can use the ATM screen.
Efficiency	Economical use of CPU, memory, storage.	App runs smoothly on a low-end phone.
Maintainability	Can be modified, corrected and extended with reasonable effort.	Adding a new bank's card support is easy.
Portability	Can run across different platforms.	WhatsApp on phone, web and desktop.

Quality models

Term	Short definition	Example
McCall's model	Early model grouping factors into product operation, revision and transition.	Maintainability falls under revision.
ISO/IEC 25010	Modern standard with 8 quality characteristics.	Reliability, security, usability, etc.
CMMI	Rates the maturity of the development process, not the product.	A company moving from ad-hoc to defined processes.

Quality Assurance

Term	Short definition	Example
QA	Proactive, process-oriented discipline that builds quality into the process so fewer defects occur.	Setting coding standards, training, audits.
QC	Detection-focused; examines the actual product to find defects.	Testing the finished app.
QA vs QC	QA = prevention, process, whole lifecycle. QC = detection, product, during/after implementation.	QA builds the road; QC inspects the cars.
QA objectives	Process-focused, proactive, standards and reviews, continuous improvement.	Lessons learned feed back into the process.
QA process (5 steps)	Define standards, train team, conduct reviews, audit process, improve continuously.	A repeating loop, never a one-time checklist.

Defect management: three layers

Term	Short definition	Example
Defect prevention	Stop defects from being introduced by fixing root causes.	Vaccine.
Defect reduction	Find and remove defects before release.	Doctor treatment.
Defect containment	Limit the impact of defects that reach production.	Quarantine.
Root cause analysis (RCA)	Trace a defect to its real cause. Steps: identify, trace to origin, categorize (people/process/tooling), correct and monitor.	Weak password accepted -> root cause: requirement never stated password length.
Prevention techniques	Requirement clarity sessions, design/code reviews, pair programming, static analysis, reusable components, process maturity (CMMI).	Peer review before merging code.
Cost of fixing defects	Cost rises the later a defect is found. L1 chart: 1, 3, 7, 15, 30 (Requirements, Design, Coding, Testing, Production).	A wrong tax rate in requirements is one line to fix; in production it needs refunds and patches.
Verification	'Are we building the product right?' Checks against specification via reviews/inspections.	Checking a tailor's stitching against measurements.
Validation	'Are we building the right product?' Checks the product meets real customer needs via testing.	Customer wanted a full suit, not just a kurta.
Testing levels (L1)	Unit, integration, system, acceptance, regression.	Function -> module pair -> whole app -> client sign-off -> re-test after change.
Defect lifecycle (L1)	New, Assigned, Fixed, Verified, Closed.	Bug logged in Jira and tracked to closure.
Defect density / leakage	Defects per unit of code / share of defects escaping to later stages.	6 defects in 2 KLOC = 3 per KLOC.

Term	Short definition	Example
Containment techniques	Fault isolation, exception handling, monitoring/logging, redundancy/failover, canary releases, rollback.	UPS during load-shedding; roll back a bad update.

Lecture 2: Quality Attributes and Quality Engineering

Term	Short definition	Example
Quality attributes (NFRs, '-ilities')	Describe HOW WELL a system does its job; main drivers of architecture.	'Transfer completes in under 2 seconds, 99.99% available, encrypted.'
Functional requirement	Describes WHAT the system does.	'Customer can transfer money.'
Three views of quality	Conformance to requirements, fitness for use, structural quality.	Works as specified, works in real conditions, code is clean.
Runtime attributes	Visible while running: performance, security, availability, usability.	Page load time, uptime.
Design-time attributes	Built into structure: maintainability, testability, modifiability, portability.	Modular code is cheaper to change.

ISO/IEC 25010: 8 characteristics

Term	Short definition	Example
Functional suitability	Does the right things, correctly.	Calculator gives 2+2=4.
Performance efficiency	Speed and resource use under load.	Site stays fast on sale day.
Compatibility	Coexists and exchanges data.	Two apps share data.
Usability	Easy, learnable, accessible.	New user needs no manual.
Reliability	Runs without failure when needed.	Banking app always available.
Security	Protects data and blocks misuse.	Others cannot open your account.
Maintainability	Easy to modify and fix.	Bug fixed in a day.
Portability	Moves across environments.	Runs on Windows and Linux.

Attributes in depth

Term	Short definition	Example
Performance metrics	Response time (e.g. under 200 ms), throughput (10k req/s), p95 latency, CPU under 70%.	p95 under 2 s: 95 of 100 users get a response within 2 s.
Performance tactics	Caching, load balancing, async processing, DB indexing, horizontal scaling.	Cache popular products.
Scalability	Vertical = bigger machine; horizontal = more machines.	Opening more counters on sale day.
Security (CIA)	Confidentiality, Integrity, Availability. Tactics: authentication/authorisation, encryption, audit logging.	Encrypted payment data.
Reliability tactics	Fault tolerance, redundancy, timeouts/retries/circuit breakers, health checks, graceful degradation.	Hide the recommendations panel if it fails; keep the shop running.
Availability formula	Availability = MTBF / (MTBF + MTTR).	MTBF 99 h, MTTR 1 h: 99/100 = 99%.
Usability aspects	Learnability, efficiency, error prevention, accessibility (WCAG).	Gmail 'Undo Send'.

Term	Short definition	Example
Maintainability aspects	Modularity, low coupling/high cohesion, readable code, living documentation.	Change stays in one module.

Quality attribute scenario (6 parts) and trade-offs

Term	Short definition	Example
Scenario parts	Source, Stimulus, Artifact, Environment, Response, Response Measure. Makes a requirement testable.	End user, 1,000 concurrent searches, search service, peak load, returns results, 95% within 2 s.
Security vs Usability	Extra checks slow users.	2FA only for sensitive actions.
Performance vs Maintainability	Optimised code is hard to read.	Optimise only measured hot spots.
Availability vs Cost	Redundancy needs extra infrastructure.	99.99% for a hospital system; 99% for a notice board.
Modifiability vs Performance	Abstraction layers add overhead.	Layer where change is likely.
Portability vs Performance	Generic code cannot use platform features.	Isolate platform code behind interfaces.
ATAM	Method to decide attribute priorities and trade-offs.	Workshop with stakeholders.

Quality Engineering (QE)

Term	Short definition	Example
Quality engineering	Building quality in across the whole lifecycle: prevention over detection, whole-team ownership, data-driven improvement.	Regular car servicing instead of repairing after breakdown.
QC vs QA vs QE	QC: finished product, reactive. QA: process, proactive. QE: whole lifecycle, engineering-driven (automation, CI/CD, metrics).	Marking papers / setting teaching standards / running the whole learning system with feedback.
Lifecycle stages	Requirements, Design, Build, Verify, Release, Operate, with feedback and RCA looping back.	Threat modelling in design; monitoring in operate.
Static vs dynamic	Static: no code executed (reviews, linters). Dynamic: code run (unit, system, load tests).	Check car papers vs test drive.
Test pyramid	Many unit, some integration, few UI/E2E. Roughly 70/20/10.	Fast unit tests catch most bugs cheaply.
Defect density	Defects per KLOC.	50 defects / 25 KLOC = 2.
DRE	(Defects found before release / total defects) x 100.	90 found, 10 escaped = 90%.
Code coverage	% of code executed by automated tests.	600 of 800 lines = 75%.
Cyclomatic complexity	Number of independent paths; lower is easier to test.	Function with 10 branches is hard to test.
MTBF / MTTR	Mean time between failures / to repair.	Used in availability.
Shift left / shift right	Test earlier (reviews, TDD, CI/CD) / monitor after release (observability, chaos testing).	Every commit built and tested automatically.
L2 cost chart	Illustrative: 1, 5, 10, 20, 100.	Same idea as L1: later is costlier.

Lecture 3: Software Testing: Concepts, Approaches and Planning

Term	Short definition	Example
Software testing	Evaluating a system to check it meets requirements and to find defects before release; investigative, reduces risk, builds confidence.	Load-testing a new bridge before opening.
Error	Human mistake producing an incorrect result.	Developer misreads a requirement.
Defect (fault/bug)	Flaw in software caused by an error.	Wrong formula sits in the code.
Failure	Actual behaviour deviates from expected.	User sees a wrong calculation result.
Chain	Error -> Defect -> Failure.	Wrong GPA formula -> bug -> wrong GPA shown.
Verification	'Right product built right?' Check work products against spec, often without running software.	Reviews, walkthroughs, static analysis.
Validation	'Right product?' Run the software to see it meets user needs.	System, UAT, beta testing.

Seven testing principles

Term	Short definition	Example
1. Presence of defects	Testing shows defects exist, never that none exist.	100 passed tests do not prove zero bugs.
2. Exhaustive testing impossible	Cannot test every input and combination.	Use EP and BVA instead.
3. Early testing	Start early; defects are cheapest to fix early.	Review requirements first.
4. Defects cluster	Few modules contain most defects.	Payment module has most bugs.
5. Pesticide paradox	Same tests stop finding new bugs; vary them.	Update test cases regularly.
6. Context dependent	Different systems need different testing.	Banking vs game.
7. Absence-of-errors fallacy	No defects found does not mean the system is useful.	Bug-free app nobody needs.

Levels and approaches

Term	Short definition	Example
Unit testing	Individual functions/classes in isolation. Owner: developer.	Test a password-check function.
Integration testing	Interaction between modules. Owner: developer/tester.	Login module with database.
System testing	Whole integrated application. Owner: test team.	Login to order placement.
Acceptance testing	Business needs from user's view. Owner: users/client.	Client approves the app.
Static testing	Code not executed; finds defects early.	Design reviews, linters.
Dynamic testing	Code executed with input.	Load and usability testing.
Black-box	Test inputs/outputs without knowing code. Techniques: EP, BVA.	Press remote buttons, watch the TV.
White-box	Test internal logic and paths. Techniques: statement, branch/path coverage.	Engineer traces the circuit.

Term	Short definition	Example
Grey-box	Partial internal knowledge guides black-box tests.	Test an API knowing the DB schema.
Functional testing	Checks WHAT the system does: unit, integration, smoke, regression.	Add to cart works.
Non-functional testing	Checks HOW WELL: performance, security, usability, compatibility.	Add to cart works with 10,000 users.
Manual testing	Human runs tests step by step; best for exploratory/usability.	Judging how a new design feels.
Automated testing	Scripts run tests; best for regression and load; high setup cost.	500 regression tests per release.

Test design techniques and re-testing

Term	Short definition	Example
Equivalence partitioning	Divide inputs into groups behaving the same; test one value per group.	Age 18-60: invalid 10, valid 30, invalid 70.
Boundary value analysis	Test at, just below and just above boundaries.	Age 18-60: 17, 18, 19, 59, 60, 61.
Decision table	List all condition combinations with expected action.	Coupon yes/no x order over $50 yes/no -> 20%, 10%, 5%, 0%.
State transition	Test valid transitions and that invalid ones are rejected.	Pending -> Approved -> Shipped; a Rejected order can never ship.
Regression testing	Re-run existing tests after changes to confirm nothing broke.	After a bug fix, checkout still works.
Smoke testing	Quick broad check that critical functions work on a new build.	App starts, login works.
Sanity testing	Narrow check that one specific fix works.	Printer prints after repair.

Test planning

Term	Short definition	Example
Test plan	Formal document of scope, objectives, approach, resources, schedule and risks of testing.	Wedding plan for testing.
Test plan benefits	Sets clear objectives, aligns stakeholders, manages risk.	Everyone knows what 'done' means.
Components (IEEE 829)	Objectives and scope; strategy; resources and roles; schedule; risks and contingencies; entry and exit criteria.	Exit: 95% pass, no critical defect open.
Planning steps	Analyse requirements, define strategy, define objectives, plan resources, build schedule, set entry/exit criteria.	Follow in this order.
Estimation methods	WBS, historical data, function/test point analysis, expert judgement.	Last app's 20 screens took 10 days -> 30 screens about 15 days.
Roles	Test manager (plan, reporting), test lead (strategy, coordination), tester (writes/executes tests, logs defects), automation engineer (scripts).	Manager, captain, players, analyst.

Lecture 4: Testing Documentation, Reporting and Historical Data

Term	Short definition	Example
Why document testing	Communication, repeatability, traceability, evidence.	New tester repeats old tests from the case document.
Test plan	What, how, who and when; written before testing, updated on scope change.	Scope, approach, tools, schedule, risks.
Test design specification	Features to test, techniques (EP, BVA, decision table), pass/fail criteria.	Use BVA for the age field.
Test case specification	ID, preconditions, inputs, expected output, dependencies.	TC-01 login test.
Test procedure	Order of running cases, with setup and cleanup.	Create test account, run, delete.
Test data	Valid, invalid and boundary values prepared in advance, kept apart from cases.	standard_user, locked_out_user.
Test script	Automated version of manual steps.	Selenium login script.
Test log	Time-ordered record of what was run, by whom, on which build.	20 Oct, Tester A, TC-06, v1.2, Fail.
Defect report	Details of one failure: ID, title, steps, expected, actual, severity/priority, status.	D-014: no error shown for empty login.
Test summary report	Final results, open defects and release recommendation.	Pass rate 95%, no critical open defects: release.
Traceability	Every test case links to a requirement ID so gaps are visible.	Requirement with no test case = coverage gap.

Test case example (SauceDemo login)

Field	Value
Test case ID	TC-01
Requirement ID	REQ-LOGIN-01
Precondition	User is registered
Steps	Enter valid email and password, click Login
Test data	standard_user and valid password
Expected result	Products page opens
Actual and status	Products page opens, Pass

Defects, severity and reports

Term	Short definition	Example
Defect life cycle	New, Assigned, Fixed, Retested, Closed. Failed retest = Reopened. Also Rejected or Deferred.	Bug fixed, retest fails, goes back to Assigned.
Severity	Impact on the system; usually set by tester.	App crashes in a rarely used report.
Priority	Urgency of fixing; usually set by manager/client.	Wrong company name on the home page: fix now.
High severity, low priority	Serious but rarely hit.	Crash in a rarely used report: fix next release.

Term	Short definition	Example
Low severity, high priority	Minor but very visible.	Wrong company name on home page.
Defect report	Raised at once for developers; must let them reproduce without questions.	Clear steps and expected vs actual.
Progress report	Daily/weekly status for managers: executed, passed, failed, blocked.	40 of 50 executed, 34 passed.
Summary report	Final results for clients/management: scope, counts, open defects by severity, risks, recommendation.	Release recommended.

Metrics and historical data

Term	Short definition	Example
Pass rate	Passed / executed.	34/40 = 85%.
Execution rate	Executed / planned.	40/50 = 80%.
Defect density	Defects / size.	12 defects / 4 KLOC = 3.
Removal efficiency	Found before release / total defects.	45/50 = 90%.
Historical data	Memory of the testing process: defects per module, effort spent, results per build, defect causes, test cases reused.	Login 6 defects, Cart 3.
Uses of historical data	Estimation, defect trends (focus on weak modules), process improvement.	Module with most defects last release gets more tests.
Good practice	Record consistently, store in one place, review after every release.	One shared defect tracker.

Tip: for 'difference' questions draw a table; for numericals write the formula first, then values, then answer.